

## Aula 2: Ambiente de Desenvolvimento e Git

Prof. Lucas Ricardo Duarte Bruzzone

17 de Fevereiro de 2025

# Agenda da Aula

- 1 Configuração do Ambiente
- 2 Introdução ao Git
- 3 Comandos Básicos
- 4 Boas Práticas
- 5 Exemplo Prático: Evolução de Código
- 6 Exercícios Práticos
- 7 Recursos Adicionais

- **Editor de Código/IDE**

- Visual Studio Code
- Sua IDE preferida para a linguagem do projeto

- **Git**

- Windows: `https://git-scm.com/download/win`
- Linux: `sudo apt-get install git`
- macOS: `brew install git`

- **GitHub Desktop** (Opcional)

- Interface gráfica para Git
- Útil para iniciantes

# Configuração Inicial do Git

```
1 # Configurar nome de usuário
2 git config --global user.name "Seu Nome"
3
4 # Configurar email
5 git config --global user.email "seu.email@exemplo.com"
6
7 # Configurar editor padrão (opcional)
8 git config --global core.editor "code --wait"
```

# O que é Git?

- Sistema de controle de versão distribuído
- Criado por Linus Torvalds em 2005
- Permite rastrear mudanças no código
- Facilita o trabalho em equipe
- Mantém histórico completo do projeto

# Conceitos Fundamentais

**Repositório** Pasta do projeto com histórico de versões

**Commit** Conjunto de alterações salvas

**Branch** Linha independente de desenvolvimento

**Merge** União de branches

**Remote** Repositório remoto (ex: GitHub)

# Iniciando um Projeto

```
1 # Criar novo repositório
2 git init
3
4 # Clonar repositório existente
5 git clone <url-do-repositorio>
```

# Ciclo Básico de Trabalho

```
1 # Verificar status das altera es
2 git status
3
4 # Adicionar arquivos ao stage
5 git add <arquivo>
6 git add . # adiciona todos os arquivos
7
8 # Criar commit
9 git commit -m "mensagem descritiva"
10
11 # Enviar altera es para reposit rio remoto
12 git push origin main
```



# Branches e Merge

```
1 # Criar nova branch
2 git branch feature-nova
3
4 # Mudar para branch
5 git checkout feature-nova
6
7 # Criar e mudar de branch (atalho)
8 git checkout -b feature-nova
9
10 # Merge de branches
11 git checkout main
12 git merge feature-nova
```

# Boas Práticas com Commits

- Faça commits pequenos e frequentes
- Escreva mensagens claras e descritivas
- Use verbos no imperativo (Add, Fix, Update)
- Exemplo: "Add user authentication feature"

# Boas Práticas com Branches

- Use branches para novas features
- Mantenha a main/master sempre estável
- Nomeie branches de forma descritiva
- Exemplos:
  - `feature/login-system`
  - `bugfix/navbar-alignment`
  - `hotfix/security-vulnerability`

# Exemplo Prático: Calculadora

- Vamos ver a evolução de um código simples
- Exemplo: Calculadora básica
- Três versões do mesmo código:
  - 1 Código monolítico (tudo no main)
  - 2 Código com funções separadas
  - 3 Código orientado a objetos com SOLID

# Versão 1: Código Monolítico

```
1 def main():
2     print("Calculadora - Vers o 1")
3     num1 = float(input("Digite o primeiro n mero: "))
4     num2 = float(input("Digite o segundo n mero: "))
5     op = input("Digite a opera o (+, -, *, /): ")
6
7     if op == '+':
8         resultado = num1 + num2
9     elif op == '-':
10        resultado = num1 - num2
11    elif op == '*':
12        resultado = num1 * num2
13    elif op == '/':
14        if num2 == 0:
15            print("Erro: Divis o por zero!")
16            return
17        resultado = num1 / num2
18    print(f"Resultado: {resultado}")
```

## Versão 2: Funções Separadas

```
1 def soma(a, b):  
2     return a + b  
3  
4 def subtracao(a, b):  
5     return a - b  
6  
7 def multiplicacao(a, b):  
8     return a * b  
9  
10 def divisao(a, b):  
11     if b == 0:  
12         raise ValueError("Divis o por zero!")  
13     return a / b  
14  
15 def calculadora_v2():  
16     # ... input do usu rio ...  
17     if op == '+':  
18         resultado = soma(num1, num2)  
19     # ... resto das opera es ...
```

## Versão 3: Classes e SOLID

```
1 class Operacao:
2     def executar(self, a, b):
3         pass
4
5 class Soma(Operacao):
6     def executar(self, a, b):
7         return a + b
8
9 class Calculadora:
10     def __init__(self):
11         self.operacoes = {
12             '+': Soma(),
13             '-': Subtracao(),
14             '*': Multiplicacao(),
15             '/': Divisao()
16         }
17
18     def calcular(self, a, b, operador):
19         return self.operacoes[operador].executar(a, b)
```

# Exercício 1: Configuração Inicial

- 1 Instale o Git
- 2 Configure seu nome e email
- 3 Crie uma conta no GitHub



## Exercício 2: Primeiro Repositório

- 1 Crie um novo repositório local
- 2 Adicione um arquivo README.md
- 3 Faça seu primeiro commit
- 4 Crie um repositório no GitHub
- 5 Conecte e envie seu repositório local

## Exercício 3: Branches

- 1 Crie uma nova branch
- 2 Faça algumas alterações
- 3 Commit das alterações
- 4 Merge com a main

## Exercício 4: Colaboração em Grupo

- 1 Com seu grupo de projeto (2-3 alunos)
- 2 Clone o repositório do projeto
- 3 Cada membro faz alterações em uma branch
- 4 Criem Pull Requests e revisem o código

- Git Documentation: <https://git-scm.com/doc>
- GitHub Guides: <https://guides.github.com>
- Git Cheat Sheet:  
<https://education.github.com/git-cheat-sheet-education.pdf>

# Atividade para Próxima Aula

- Com seu grupo já definido (2-3 alunos):
- Criar repositório para o projeto da disciplina
- Configurar estrutura básica conforme o projeto escolhido
- Adicionar membros do grupo como colaboradores
- Cada membro deve criar uma branch e fazer um commit

# Referências

-  Chacon, S. and Straub, B. (2014). Pro Git. Apress.
-  GitHub Docs. <https://docs.github.com>
-  Atlassian Git Tutorial. <https://www.atlassian.com/git/tutorials>